Triggering Generalist Reasoning via Predictive Uncertainty for Dual-System VLA

2026. 10. 27

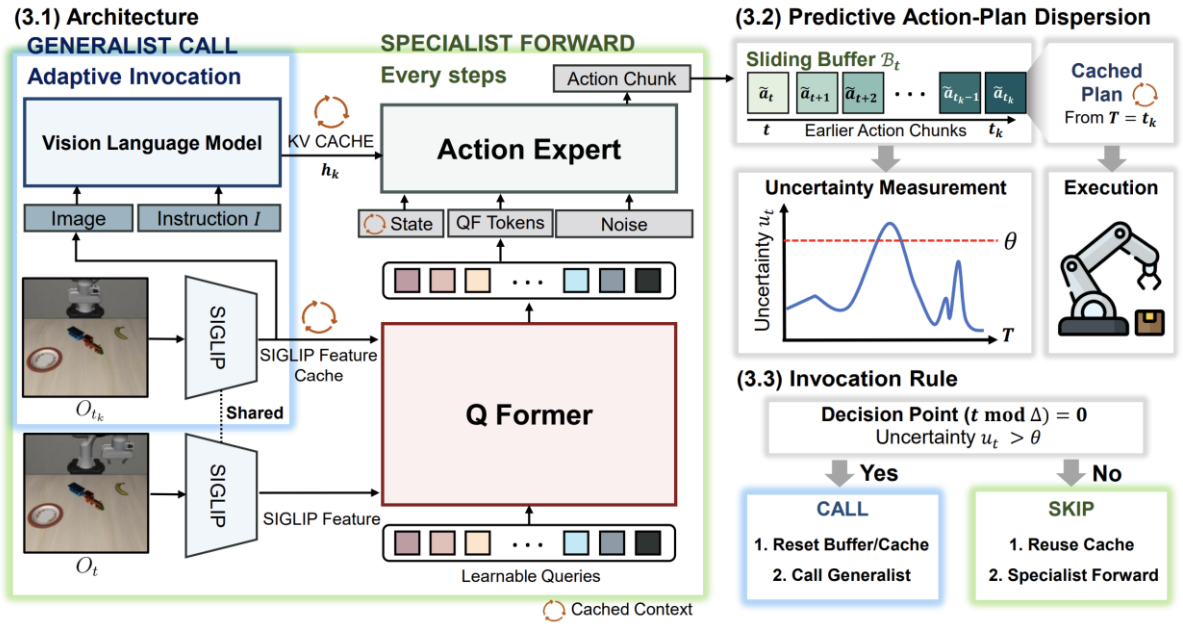
Seonyeong Jeong

Covered Papers

Triggering Generalist Reasoning via Predictive Uncertainty for Dual-System VLA (NeurIPS 2026)

Hyemin Yang*, Wooseong Jeong*, Giwon Lee*, Kuk-Jin Yoon

Dual-System VLA, Predictive Uncertainty, Efficient Robotic Control



Introduction

Dual-System VLA

- 대규모 VLM 기반 VLA의 추론 비용으로 실시간 로봇 제어에 제약
- 느린 Generalist와 빠른 Specialist를 분리하여 추론 능력과 제어 속도 확보

When to Invoke the Generalist?

- 기존 방식은 **static frequency**로 Generalist 호출
- 동일 rollout 내에서도 장면 변화와 조작 난이도가 지속적으로 달라짐

  → 쉬운 구간에서는 불필요한 호출, 예상치 못한 변화에는 재추론 지연 발생

TUD 제안

- 행동 재예측의 불일치를 이용한 **Generalist 호출 시점 결정**
- 별도의 uncertainty model이나 수작업 phase annotation 없이 호출 여부 판단

Problem Formulation: Dual-System VLA

Generalist & Specialist

- **Generalist** (System 2): 언어 지시와 관측을 바탕으로 **고수준 context 생성**
- **Specialist** (System 1): 최신 관측과 Generalist의 context을 이용하여 **action 생성**

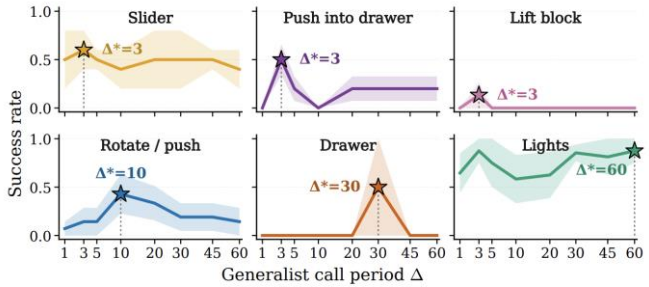
Task-Dependent Call Interval

- OpenHelix에서 task별 최적 호출 간격(Δ)이 크게 달라짐
- Slider, Push, Lift는 짧은 간격(Δ=3), Lights는 긴 간격(Δ=60)에서 높은 성공률

➡ 모든 task에 공통으로 적합한 고정 호출 간격 부재

Success–Efficiency Trade-off

- RoboDual은 호출 간격 증가 시 long-horizon 성능 급감
- 반대로 빈번한 Generalist 호출은 추론 지연 증가
- Adaptive 호출을 위한 조건: Interval-flexible architecture와 저비용 online signal



(a) OpenHelix on CALVIN: best $\Delta^\star$ varies across tasks.

Δ	Tasks completed in a row (%) ↑					Avg. Len. ↑
	1	2	3	4	5	
2	96.0	82.0	71.0	64.0	51.0	3.64
4	92.0	79.0	65.0	60.0	53.0	3.49
8	95.0	74.0	58.0	51.0	44.0	3.22
12	69.0	40.0	15.0	4.0	0.0	1.28
16	59.0	28.0	12.0	3.0	0.0	1.02
20	47.0	20.0	8.0	0.0	0.0	0.75

(b) Robodual on CALVIN at varying Δ.

Problem Formulation: Adaptive Generalist Calling

Periodic Call/Skip Decision

- 일정한 decision point마다 invocation policy가 **CALL / SKIP** 선택
- **CALL**: Generalist를 호출하여 context 갱신
- **SKIP**: 기존 context를 유지하고 다음 decision point까지 호출 연기
- Decision point의 주기는 고정하되, 실제 Generalist 호출 간격은 가변적

Objective

- Task success를 높이면서 Generalist 호출 횟수 최소화
- λ는 성공률과 호출 비용 사이의 trade-off 조절

$$\pi^{\star} \;=\; \arg\max_{\pi} \; \mathbb{E}[\,\mathrm{SR}(\pi)\,] \;-\; \lambda \cdot \mathbb{E}[\,N_{\mathcal{G}}(\pi)\,]$$

Method: Architecture

π_0-Based Dual-System Architecture

- Single-system π_0에 **Q-Former bridge**를 추가하여 Generalist와 Specialist 분리
- Q-Former가 현재·캐시된 SigLIP feature를 함께 참조하여 conditioning token 생성

Generalist Call

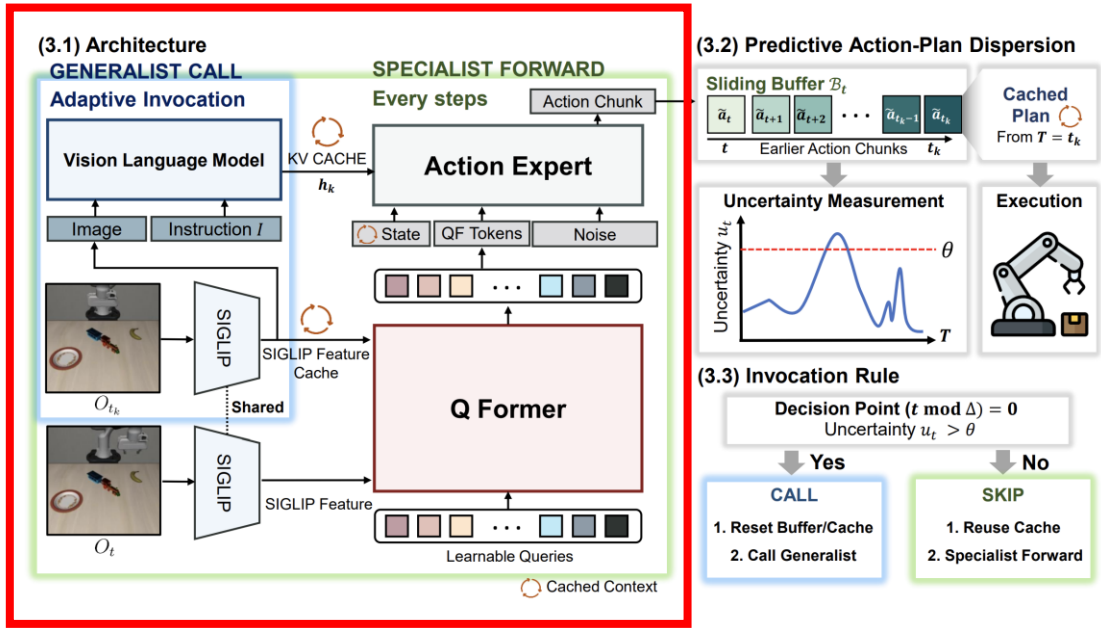
- 현재 관측과 언어 지시를 처리하여 **VLM KV cache**와 **SigLIP feature** 갱신
- 갱신된 문맥으로 실제 실행할 action chunk 생성

Specialist Forward

- 최신 이미지와 캐시된 문맥을 이용하여 action chunk 재예측
- VLM의 language-image token은 재계산하지 않음

Action Execution

- 호출 사이에는 마지막 Generalist 호출 시 생성한 chunk를 순차 실행
- 중간 재예측은 uncertainty 측정에 사용하며, **실행 chunk를 매번 교체하지 않음**



Method: Predictive Action-Plan Dispersion

Cross-Step Re-Predictions

- 동일한 Generalist context 아래에서 관측이 갱신될 때마다 action chunk 재예측
- 최근 E개의 재예측을 sliding buffer에 저장
- Generalist 재호출 시 buffer 초기화

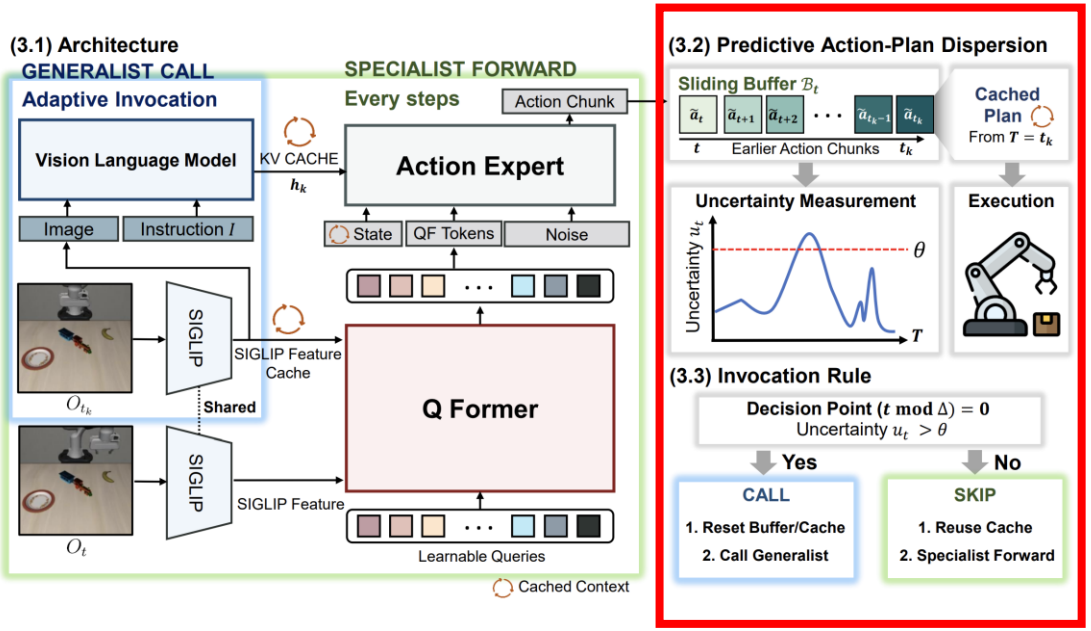
Predictive Action-Plan Dispersion

- 각 재예측에서 **동일한 실행 예정 slot**의 action을 비교
- Action dimension별 표준편차를 계산한 뒤 평균하여 uncertainty 산출
- 낮은 dispersion: 최근 재예측 간 일치
  높은 dispersion: 관측 변화에 따라 행동 계획이 지속적으로 변경됨

Low-Cost Signal

- 동일 관측에 대한 반복 sampling 없이 시간에 따른 재예측 활용
- 기본 설정: buffer 크기 E = 5, 재예측 측정 간격 2 steps

$$u_t = \frac{1}{d_a} \sum_{d=1}^{d_a} \mathrm{std}_{i=0,\ldots,E-1}\left(\tilde{a}_{t-i,\, j(t),\, d} \right).$$



Method: Invocative Rule

Action-Deviation Bound

- 캐시된 action과 현재 재예측 action의 차이를 두 항으로 상한화
- **Dispersion term**: 최근 buffer 내부의 예측 불일치
- **Drift term**: 초기 anchor가 buffer에서 빠진 이후 누적된 관측 변화 반영

$$\left\|\tilde{a}_{t_k}[j(t)] - \tilde{a}_t[j(t)]\right\|_2 \;\le\; \alpha_E\, u_t \;+\; \beta\, L_{\mathcal{S}}\, \rho_{t,k},$$

Call-or-Skip Rule

- 마지막 호출 이후 uncertainty의 **누적 평균**이 임계값보다 낮고, 최대 호출 간격(Δ_{max})에 도달하지 않은 경우 SKIP
  그 외에는 CALL하여 context, 실행 chunk 갱신 및 buffer 초기화
- Dispersion은 임계값으로, 장기간의 context 재사용은 최대 호출 간격으로 제한

Online Threshold Adaptation

- 최근 uncertainty 분포의 p-th percentile을 임계값으로 사용
- 높은 p는 더 많은 SKIP을 허용하는 설정
- 기본 설정: **p70**, decision period **3 steps**, maximum interval **20 steps**

Experiments: Experimental Setup

Backbone & Training

- π_0 checkpoint에 Q-Former bridge를 추가하여 fine-tuning
- 30,000 training steps, batch size 32, NVIDIA H200 4장
- Action chunk horizon 50, 실행 action dimension 7

Benchmarks

- **VLA-Arena**: Safety, Distractor, Extrapolation, Long Horizon의 4개 domain, 11개 task suite
- L0에서 학습 후 L0 성능 및 L1 일반화 평가
- **CALVIN ABC→D**: 미관측 환경(D)의 연속 task 수행 평가

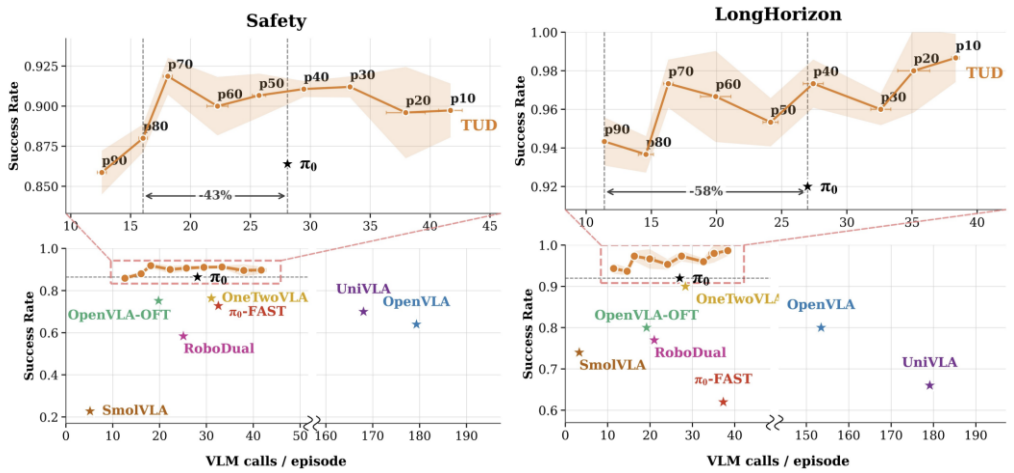
Evaluation Metrics

- **Success Rate (SR)**: task 성공률
- **Cumulative Cost (CC)**: 충돌 등 안전 제약 위반의 누적 비용
- **Calls / Episode**, **Wall-clock Time**: Generalist 호출 횟수와 실행 시간

Experiment: Cost-Success Trade-off

Operating Curve

- Percentile p를 조절하여 SR과 Generalist 호출 횟수의 operating curve 구성
- 서로 다른 backbone 간 절대 성능 순위보다 **호출 비용 대비 성공률**에 초점



VLA-Arena Results

- **Safety**: p70에서 π_0와 동등한 SR을 유지하며 호출 **43% 감소**
- **Long Horizon**: 높은 SR을 유지하며 π_0 대비 호출 **58% 감소**
- Task별 최적 p는 달라질 수 있으며, 모든 task에서 단일 설정이 최고 성능을 보이는 것은 아님

Inference Latency

- H200 기준 Generalist **105.91 ms**, Specialist **13.53 ms**
- Specialist forward 비용은 Generalist call의 약 **1/7.8**

Table 3: Per-call latency.

Forward	ms ↓	Hz ↑
Generalist	105.91	9.44
Specialist	13.53	73.91

Experiments: Signal Validation

Matched Call Budgets

- 동일 invocation rule에서 uncertainty signal만 교체
- Sampling 기반 uncertainty, diffusion loss, visual feature distance 등과 비교
- 높은 호출 budget: **17.8 calls / SR 86.7% / 9.5 s**
  낮은 호출 budget: **9.0 calls / SR 82.8% / 7.9 s**
- 두 budget 구간 모두 비교 신호 중 가장 높은 성공률과 가장 짧은 실행 시간 달성

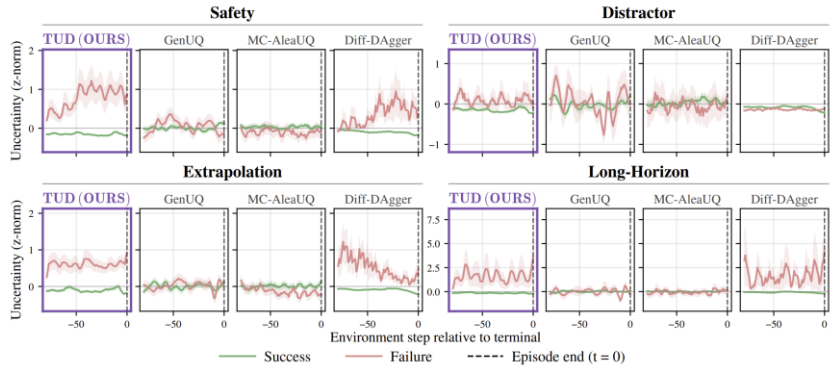
Table 4: VLM re-invocation criteria at matched call budgets on VLA-Arena L0.

Method	Calls	SR (%) ↑	Wall (s) ↓
High budget (≈17–18 calls/ep)			
Diff-DAgger	16.5	83.7	31.1
BayesDiff	16.3	83.5	11.9
Visual	17.2	83.0	11.8
AAC-ent.	16.4	82.7	28.7
MC-AleaUQ	19.1	80.2	11.6
GenUQ	17.3	74.7	29.7
TUD	17.8	**86.7**	**9.5**

Method	Calls	SR (%) ↑	Wall (s) ↓
Low budget (≈8–10 calls/ep)			
Diff-DAgger	8.3	80.7	31.0
BayesDiff	6.8	77.0	11.8
Visual	9.3	79.8	11.3
AAC-ent.	10.1	77.2	27.7
MC-AleaUQ	8.3	73.5	10.8
GenUQ	8.9	73.8	30.4
TUD	9.0	**82.8**	**7.9**

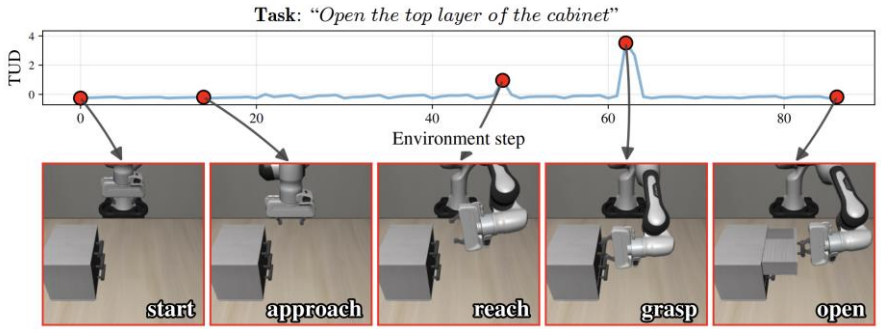
Failure-vs-Success Separation

- 비교한 신호 중 TUD만 4개 domain 모두에서 성공·실패 rollout의 일관된 분리 확인
- 실패 rollout의 종료 직전 구간에서 uncertainty 상승 경향



Per-Rollout Dynamics

- u_t가 단순 접근 구간에서는 낮고, 손잡이 탐색, grasp처럼 정밀한 조작 구간에서 상승



Experiment: Real Robot Experiments

Real Robot Setup

- SO-101 6-DoF arm, third-person/gripper-view RGB camera 2대 사용
- Pen placement, drawer closing, cube stacking의 3개 task
- Task별 teleoperated demonstration 100개로 학습, 방법별 총 30회 평가

TUD p90 vs. Fixed Δ = 9

- 성공률: **46.7% → 83.3%**
- Generalist 호출: **48.5 → 12.1회**, 약 **75% 감소**
- 실행 시간: **76.4 → 38.5초**, 약 **50% 감소**

Adaptive Invocation

- 고정 간격을 늘리면 호출 횟수와 함께 성공률도 감소
- TUD는 모든 평가 설정에서 baseline보다 높은 성공률과 적은 호출 횟수 달성

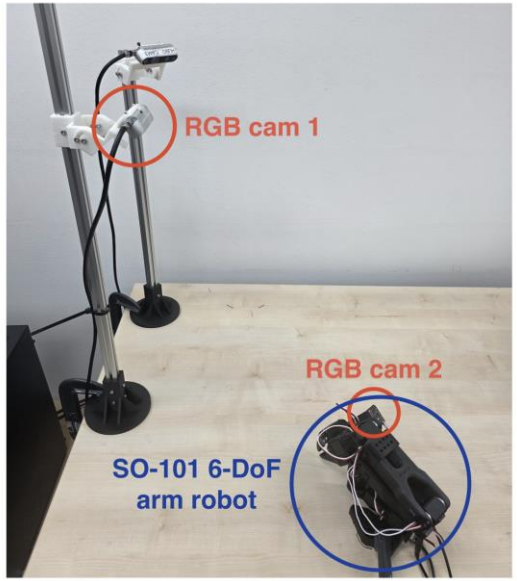


Figure 10: **Real robot setup.** SO-101 6-DoF arm with third-person and gripper-view RGB cameras.

Table 5: **Real robot results.** SR over 30 roll-outs per method (10 per task). Calls and Wall (s) are per-episode generalist invocations and execution time; failures count as 120 s.

Method	SR ↑	Calls ↓	Wall (s) ↓
Single-system π_0	0.433	49.0	79.5
Fixed $\Delta = 9$	0.467	48.5	76.4
Fixed $\Delta = 25$	0.433	34.6	76.1
Fixed $\Delta = 50$	0.400	25.9	79.2
TUD p50	0.767	24.5	53.4
TUD p70	0.733	20.8	53.5
TUD p90	**0.833**	**12.1**	**38.5**

Conclusion

Adaptive Generalist Calling

- Dual-system VLA에서 Generalist 호출 시점을 조절하는 TUD 제안
- Specialist의 시간적 재예측 불일치를 활용하여 문맥 갱신 필요성 판단

Predictive Action-Plan Dispersion

- 신호 계산을 위한 별도 모델 학습이나 반복 stochastic sampling 불필요
- Percentile 조절을 통해 성공률과 호출 비용 사이의 operating point 선택 가능

Main Findings

- 시뮬레이션과 실제 로봇에서 성공률을 유지하거나 높이면서 Generalist 호출 감소
- 추론 효율화를 위해 **언제 Generalist를 호출할 것인가**를 별도 설계 요소로 제시

Stated Limitation

- 원하는 latency budget이나 최소 성공률을 직접 만족하도록 p를 최적화하는 방식은 미제공

Idea & Discussion

Learnable Maximum Invocation Interval

- **Limitation**: 고정된 최대 호출 간격으로 drift에 대응하므로, 상황별 변화 속도 반영에 한계
- **Motivation**: 예측이 점진적으로 변하는 경우, 낮은 dispersion에도 초기 계획과의 차이가 누적되지 않을까?
- **Idea**: 관측 변화와 예측 추세에 따라 최대 호출 간격을 조절하는 정책 학습
  - 변화가 누적되는 구간에서는 간격 단축, 안정적인 구간에서는 간격 연장

고정된 buffer 크기와 재예측 간격 (ppt p.7)

- **Motivation**: buffer 크기를 5로, 재예측 간격을 2 steps로 고정하면 generalist 호출을 static frequency로 하는 것만큼 rigid하지 않을까?
- **Appendix C에서의 검증 →**

Factor	Setting	SR ↑	Calls ↓
Buffer E	3	0.832	27.3
	5	**0.850**	**21.6**
	7	0.850	18.5
Measurement Δ_s	1	0.848	28.0
	2	**0.850**	**21.6**
	4	0.852	51.2
Decision Δ	2	0.840	29.4
	3	**0.850**	**21.6**
	5	0.793	15.7
Maximum interval $\Delta_{\max}$	12	0.858	24.0
	20	**0.850**	**21.6**
	32	0.830	20.1

Thank You